EECS 388

Introduction to Computer Security

Lecture 6:

Key Exchange and Public-Key Cryptography

September 17, 2026
Prof. Halderman



Review: AEADs for Integrity+Confidentiality

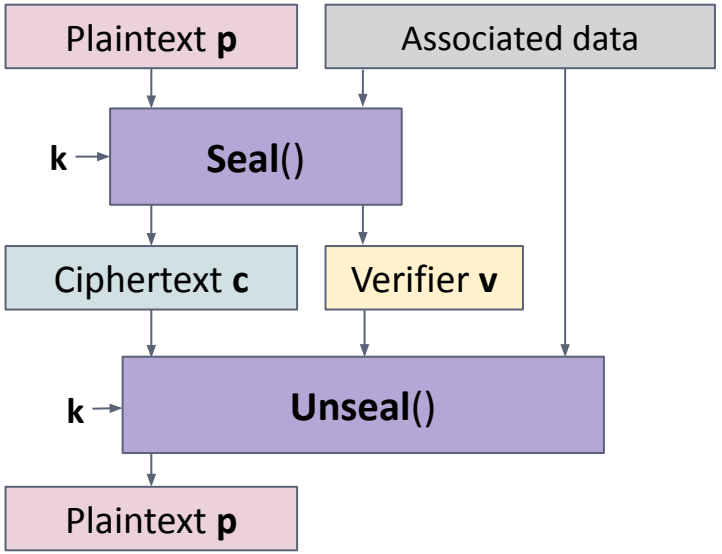
Preferred approach to integrity+confidentiality:

Authenticated encryption with associated data (AEAD)

Integrity and encryption in a single primitive:

> **c**, **v** := **Seal**(**k**, **p**, [**associated_data**])
> encrypts plaintext **p** and returns ciphertext **c** and a verifier **v** (called a "**tag**")
>
> **p**, **err** := **Unseal**(**k**, **c**, **v**, [**associated_data**])
> returns **p** or an error if **v** does not match the supplied **c** and **associated_data**

Optional **associated_data** is covered by verifier but *not encrypted*.



Examples: AES-GCM, ChaCha20-Poly1305

We still need: How can Alice and Bob *share a key in the first place*?
How can Alice and Bob *tell they're talking to each other*?

Diffie-Hellman Key Exchange

Issue: How do we get a shared key?

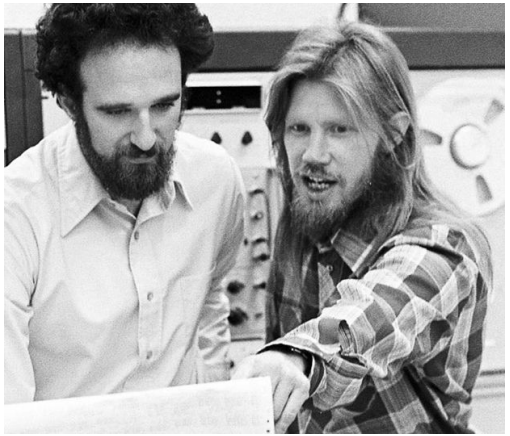
***Amazing fact*:** Alice and Bob can have a *public conversation* to derive a secret shared key!

Diffie-Hellman (D-H) key exchange

The birth of modern cryptography

1976:
Whit Diffie and Martin Hellman "New Directions in Cryptography"

(Earlier, in secret, by Malcolm Williamson at GCHQ)

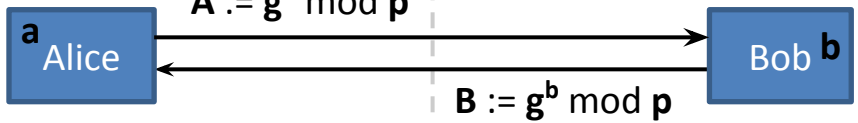


Turing Award in 2015

1. Use public (often from standards) parameters:
   p: a large prime (say, 2048-bits; *some restrictions)
   g: a primitive root modulo **p** (usually small: 2, 3…)

2. **Alice**
   Generates random secret value **a**
   (0<**a**<**p**)

   Bob
   Generates random secret value **b**
   (0<**b**<**p**)



3. Computes
   s := $\mathbf{B}^{\mathbf{a}}$ mod **p**

   Computes
   s := $\mathbf{A}^{\mathbf{b}}$ mod **p**

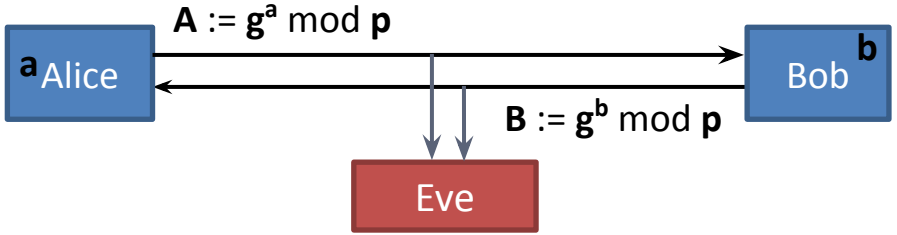
Alice and Bob each obtain the same value:

$\mathbf{B}^{\mathbf{a}} \bmod \mathbf{p} = \mathbf{g}^{\mathbf{ba}} \bmod \mathbf{p} = \mathbf{g}^{\mathbf{ab}} \bmod \mathbf{p} = \mathbf{A}^{\mathbf{b}} \bmod \mathbf{p}$

Today we usually compute D-H over **elliptic curves**, but same concept as this original finite-field version.

Security of Diffie-Hellman

Passive eavesdropping: Fails



Eve knows: **p**, **g**, $g^a \bmod p$, $g^b \bmod p$

Eve wants to compute: $s := g^{ab} \bmod p$

(This is called the **computational Diffie-Hellman problem**)

Best known approach: Find **a** or **b**, then compute **s**

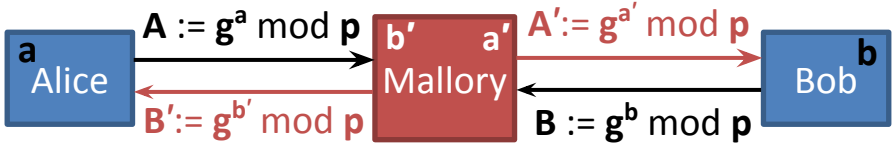
Finding **x** given $g^x \bmod p$ is an instance of the **discrete logarithm problem**

Best known algorithm: **number field sieve**

superpolynomial but subexponential in |**p**|

Believed (hoped) intractable for 2048-bit **p**

Active MITM attack: Succeeds!



Alice does D-H, *really with Mallory*,
ends up with $u := g^{ab'} \bmod p$

Bob does D-H, *really with Mallory*,
ends up with $v := g^{a'b} \bmod p$

Alice and Bob think they're talking with the other, but really they've each computed a separate secret key **shared with Mallory**

D-H gives you a shared secret,
but you don't know who it's shared with!

(We'll fix that using digital signatures…)

Issues About Keys

Defending D-H from MITMs

Approaches:

1) **Trust-on-first-use**
   Hope there's no attacker the first time, but detect if keys change later (e.g., **SSH**)
2) **Have users communicate out-of-band**
   Ask people to meet or call each other to verify the have the same **k** (e.g., **Signal**)
3) **Rely on proximity to limit MITMs**
   Attacker must be nearby, or in contact (e.g.: Many **IoT devices** attempt this)
4) **Use digital signatures**
   What HTTPS websites do. More later…

Why use D-H at all if we have other methods (as we'll see) of sending a key confidentially?

Important goal: Forward Secrecy

We usually want to communicate **repeatedly**

Pitfall: Adversary could record all the ciphertexts, then later steal our key

Solution: For each session, use D-H to generate a temporary **session key**. Use separate **long-term key** to authenticate it

Benefits: Compromising long-term key would allow future impersonation but not retrospective decryption (assuming attacker can't break D-H)

RSA: Public-key Cryptography

Since antiquity, encryption key = decryption key

"**symmetric key crypto**"

What if Alice publishes data to the world and wants everyone to verify integrity… [Publish a MAC key?]

What if Bob wants anybody to be able to send him data, confidentially… [Why is this impractical?]

Breakthrough: Keys can be distinct.

Public-key crypto

1977: RSA Cryptosystem
Ron Rivest, Adi Shamir, Leonard Adleman
(Earlier, in secret, by Clifford Cocks at GCHQ)

Turing Award in 2002



"Textbook" RSA

Key generation (in secret):

1. Generate large random primes, **p** and **q** (say, 2048 bits each)
2. Compute "modulus" **N** := **pq** (say, 4096 bits)
3. Pick small "encryption exponent" **e** (say, 65537)
   Must be relatively prime to (**p** – 1)(**q** – 1)
4. Compute "decryption exponent" **d** such that **ed** ≡ 1 mod (**p** – 1)(**q** – 1)

Yields **RSA key pair**: **Public key**: (**e**, **N**)
Private key: (**d**, **N**)

Public-key encryption !!!
Encrypt: $c := m^e \bmod N$
Decrypt: $m := c^d \bmod N$

Digital signatures !!!
Sign: $s := m^d \bmod N$
Verify: $m \stackrel{?}{=} s^e \bmod N$

Facts about RSA

Subtle fact: RSA can be used for confidentiality and/or integrity+authenticity

Confidentiality: Public-key encryption

Alice encrypts **m** using Bob's public key to get **c**

Bob decrypts **c** using Bob's private key to get **m**

Integrity/sender authenticity: Digital signatures

Alice signs **m** using Alice's private key to get **s**

Bob verifies **m**,**s** using Alice's public key

Both properties: Use both, with two key pairs

Alice encrypts **m** using Bob's public key to get **c**, then Alice signs **c** using Alice's private key to get **s**

Bob verifies **c**,**s** using Alice's public key, then Bob decrypts **c** using Bob's private key to get **m**

Is RSA secure?

Best known* way to compute **d** from **e** is factoring **N** into **p** and **q**.

Best known* algorithm: **number field sieve** (again)

superpolynomial but subexponential in |**N**|

Believed (hoped) intractable for 4096-bit **N**

But: Range of subtle mathematical attacks

RSA drawbacks: >1000X slower than AES

Messages must be shorter than **N**

Keys must be relatively large [Why?]

Elliptic-curve cryptography (ECC)

Elliptic curves are a mathematical alternative for constructing key exchange, signing, encryption

Pros: Attacks (hoped) harder, so key can be shorter

Examples: **ECDH**, **ECDSA**, **Ed25519**

Subtle Attacks on Textbook RSA

Textbook RSA Encryption

Encrypt: $\mathbf{c} := \mathbf{m}^{\mathbf{e}} \bmod \mathbf{N}$ Decrypt: $\mathbf{m} := \mathbf{c}^{\mathbf{d}} \bmod \mathbf{N}$

Small *e* attack:

If $\mathbf{e} = 3$ and $\mathbf{m} < \mathbf{N}^{1/3}$: $\mathbf{m} = \mathbf{c}^{1/3}$ (No mod! Cube root is fast)

Stereotyped message attacks:

We can efficiently compute up to a 1/**e**-fraction of the bits of an RSA-encrypted message with public exponent **e** if we know the rest of the plaintext.

Ciphertext malleability:

RSA is homomorphic under multiplication.

From $\mathbf{c} = \mathbf{m}^{\mathbf{e}} \bmod \mathbf{N}$, attacker can forge encryption of **xm**: $\mathbf{c'} := \mathbf{x}^{\mathbf{e}}\mathbf{c} \bmod \mathbf{N} = \mathbf{x}^{\mathbf{e}}\mathbf{m}^{\mathbf{e}} \bmod \mathbf{N} = (\mathbf{xm})^{\mathbf{e}} \bmod \mathbf{N}$

Key generation failures:

Suppose Alice and Bob use a bad RNG, generate moduli *with the same* ***p***: $\mathbf{N}_1 = \mathbf{pq}_1$, $\mathbf{N}_2 = \mathbf{pq}_2$.

Eve easily learns both private keys: $\mathbf{p} := \text{GCD}(\mathbf{N}_1, \mathbf{N}_2)$

Textbook RSA Signatures

Sign: $\mathbf{s} := \mathbf{m}^{\mathbf{d}} \bmod \mathbf{N}$ Verify: $\mathbf{m} \stackrel{?}{=} \mathbf{s}^{\mathbf{e}} \bmod \mathbf{N}$

Forging signatures on random messages:

When there are no constraints on messages, can trivially forge *some* messages with valid signatures.

Attacker picks random **s**, does $\mathbf{m} := \mathbf{s}^{\mathbf{e}} \bmod \mathbf{N}$.

Producing an arbitrary message with a valid signature doesn't prove sender knows private key!

Forging signatures on specific messages:

Suppose Bob will sign any message Mallory provides that he deems *non-sensitive*.

Mallory can trick Bob into signing *sensitive* **m'**:

Mallory computes: $\mathbf{m} := \mathbf{r}^{\mathbf{e}}\mathbf{m'}$ for random **r**

Bob signs **m**: $\mathbf{s} := \mathbf{m}^{\mathbf{d}} \bmod \mathbf{N} = (\mathbf{r}^{\mathbf{e}}\mathbf{m'})^{\mathbf{d}} \bmod \mathbf{N}$

Then, Mallory finds signature $\mathbf{s'} = (\mathbf{m'})^{\mathbf{d}} \bmod \mathbf{N}$ by computing $\mathbf{s}(\mathbf{r}^{-1}) \bmod \mathbf{N} = (\mathbf{r}^{\mathbf{e}}\mathbf{m'})^{\mathbf{d}}(\mathbf{r}^{-1}) \bmod \mathbf{N}$.

RSA in Practice

Textbook RSA is dangerously insecure, but we can use it as a basis for better constructions:

Safely encrypting with RSA

Encrypt a message **m** to RSA public key (**e,N**):

Use RSA to encrypt a random **x** < **N**, use a PRF to derive a key **k** from **x**, then encrypt message using a symmetric cipher and **k**.

1. Generate random **x** < **N**
2. $\mathbf{c}_1 := \mathbf{x}^{\mathbf{e}} \bmod \mathbf{N}$
3. **k** := PRF(**x**)
4. $\mathbf{c}_2 :=$ AES-GCM$_{\mathbf{k}}$(**m**)

Advantages:

- Identical messages yield different ciphertexts
- Don't have to worry about RSA padding
- Don't have to worry about message length

Safely signing with RSA

Sign a message **m** with RSA private key (**d**,**N**):

Use RSA to sign a *carefully padded* version of the digest of **v**. **Many gotchas!** A good padding scheme is "Probabilistic Signature Scheme" (PSS).

1. **v** := SHA-256(**m**)
2. **x** := PSS(**v**)
3. $\mathbf{s} := \mathbf{x}^{\mathbf{d}} \bmod \mathbf{N}$

Verifier checks padding:

1. **v'** := SHA-256(**m'**)
2. $\mathbf{x}' := \mathbf{s}'^{\mathbf{e}} \bmod \mathbf{N}$
3. Verify that **x'** is correct PSS-padded **v'**

Caution

If you don't correctly verify the padding, attacker can forge signatures w/ **Bleichenbacher's attack**

You'll exploit in Project 1!

In practice, you should almost always use **crypto libraries** to get such details right…

Quantum Computers and Public Key Crypto

D-H and RSA rest on math problems that are believed to be hard for **classical computers**: discrete log and factoring.

These problems are known to be much easier for ***quantum computers*.**

Shor's algorithm lets a QC solve factoring and DL in polynomial time!

QCs can break: RSA, D-H, ECC

Fortunately, symmetric crypto is not known to be threatened by QCs.

Grover's algorithm gives only sqrt speedup. Use larger keys (e.g., AES-256), keep AEADs

So far, nobody has built a **cryptographically relevant quantum computer (CRQC)**
(loads of engineering challenges)

… but, **record now, decrypt later attacks**
(adversary waits until CRQCs available)

… and recent progress suggests we must be prepared for CRQCs ***as soon as 2029!***

Post-quantum cryptography (PQC)

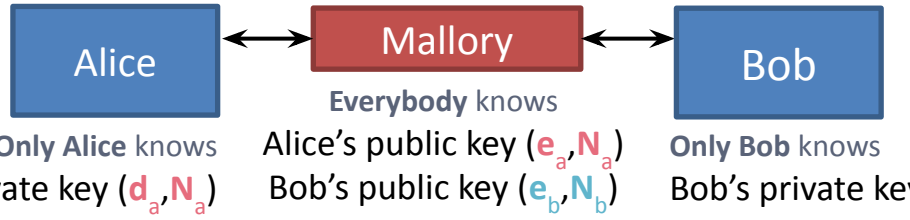
Quantum-secure (we hope!) public key algorithms, often based on **lattice problems**

e.g.: **KL-MEM** key exchange **ML-DSA** signatures

Browsers starting to ship **PQC-hybrid crypto**: securely combines ECC and lattice methods, remains secure as long as either is unbroken

A Secure Channel Protocol

Putting it all together…



1. Use **Diffie-Hellman** to generate a shared secret

Generates random a
$k := B^a \bmod p$
$A := g^a \bmod p$
$B := g^b \bmod p$
Generates random b
$k := A^b \bmod p$

2. Use **RSA signatures** to confirm we're really talking to each other (*sender authenticity*)

$x :=$ PSS(SHA-256(A,B))
Verifies that $(s_b)^{e_b} \bmod N_b$ is correct PSS-padded x
$s_a := x^{d_a} \bmod N_a$
$s_b := x^{d_b} \bmod N_b$
$x :=$ PSS(SHA-256(A,B))
Verifies that $(s_a)^{e_a} \bmod N_a$ is correct PSS-padded x

3. Derive symmetric keys for each direction using a **PRF**

$k_a := \text{HMAC-SHA256}_k(\text{“}\rightarrow\text{”})$
$k_b := \text{HMAC-SHA256}_k(\text{“}\leftarrow\text{”})$
$k_a := \text{HMAC-SHA256}_k(\text{“}\rightarrow\text{”})$
$k_b := \text{HMAC-SHA256}_k(\text{“}\leftarrow\text{”})$

4. Use **AEAD** (e.g., **AES-GCM**) or **encrypt-then-MAC** messages (*integrity* and *confidentiality*)

$c_a, v_a :=$ Seal(k_a,“Hi Bob!”)
$m_b :=$ Unseal(k_b, c_b, v_b)
c_a, v_a
c_b, v_b
$m_a :=$ Unseal(k_a, c_a, v_a)
$c_b, v_b :=$ Seal(k_b,“Hi Alice!”)

* Please don't actually code anything like this. Just use TLS!

Coming Up

Reminders:

Project 1, Part 1 due TODAY at 6 p.m.

Project 1, Part 2 due next Thursday at 6 p.m.

Quiz on Canvas after every lecture

Tuesday

The Web Platform

Intro to the Web platform

HTTP, cookies, Javascript, etc.

Thursday

Web Attacks and Defenses

XSS, CSRF, and SQLi

attacks and defenses